Reliable Candidate Selection for Budget-Constrained Machine Learning Agents

DATS-SHU-200 Data Mining
Homework 3: Baseline Reproduction Draft

1 Introduction

Large language model (LLM) agents can automate parts of machine learning engineering, including writing training code, debugging failures, and improving existing solutions. A central difficulty is deciding which proposed solution deserves an expensive execution. For example, an agent may generate several modifications to a training pipeline, while only a small number can be trained and evaluated within the available time. Executing every proposal consumes resources; discarding proposals prematurely can remove the best solution. This project studies how an agent can use imperfect predictions of candidate quality to make better selection decisions under a limited comparison budget.

Existing agent systems provide two complementary approaches to this problem. AIDE formulates machine learning engineering as tree search over code solutions: the agent drafts, debugs, and improves programs using execution feedback (Jiang et al., 2025). This makes observed performance available for subsequent decisions, but requires running candidate programs. ForeAgent instead introduces a *Predict-then-Verify* loop (Zheng et al., 2026). During improvement, it generates multiple candidates and asks an LLM to compare their code using the task description and a data analysis report. A confidence-gated selection procedure then filters candidates before execution. This approach reduces reliance on running every proposal while retaining execution feedback for the candidates that are actually verified.

However, a useful pairwise judgment does not automatically produce a reliable decision among many candidates. Judgments can conflict, and one mistaken comparison can eliminate a strong candidate. Confidence expresses the model's certainty about a particular comparison; it is neither a candidate's performance score nor a guarantee of correctness. Consequently, the rules for collecting and combining pairwise evidence are a distinct part of the agent's decision process. This motivates the research question: *With the candidate pool and comparison feedback held fixed, can a different aggregation rule select better machine learning solutions than ForeAgent's confidence-gated tournament?* A subsequent extension would examine which comparisons to request under a shared budget. Neither improvement is assumed in advance.

The baseline for this project is ForeAgent's candidate-selection component. Its released agent trajectories support a direct check of the original control logic using historical candidate batches and observations. Section 3 reports this conditional replay, establishing an implementation baseline for later aggregation experiments. The public Preference Corpus, containing 18,438 candidate pairs across 26 tasks, also supplies recorded execution scores for subsequent selection-quality evaluation (Zheng et al., 2026). MLE-bench provides the broader machine learning engineering task setting (Chan et al., 2024). Historical-output agreement, prediction accuracy, and end-to-end task performance each supply a different level of evidence.

2 Baseline Explanation

2.1 Problem formulation and notation

Let I denote a task description, D its dataset, and C a candidate program implementing a machine learning workflow. Let $M(C; I, D)$ denote its task-specific evaluation metric after execution. Define an oriented utility $u(C)$ so that higher values are always better: $u(C) = M(C; I, D)$ for a maximized metric such as AUC, and $u(C) = -M(C; I, D)$ for a minimized metric such as log loss. The underlying engineering objective is to find

$$C^* \in \arg\max_{C \in \mathcal{C}} u(C), \tag{1}$$

where $\mathcal{C}$ is the space of possible programs. Because obtaining $u(C)$ requires execution, ForeAgent predicts relative quality before spending this cost. This notation describes the optimization objective; the true utility is unavailable to the predictor at decision time.

For candidates C_i and C_j, the comparison input consists of I, a data analysis report D_{rep}, the two programs, and a prompt P. A pretrained LLM with fixed parameters θ returns

$$(z_{ij}, \widehat{y}_{ij}, c_{ij}) = f_\theta(I, D_{\text{rep}}, C_i, C_j; P), \tag{2}$$

where z_{ij} is the generated reasoning, $\widehat{y}_{ij} \in \{0, 1\}$ identifies the preferred input position, and $c_{ij} \in [0, 1]$ is the reported confidence. The evaluation label identifies which candidate achieved the better recorded score. The index is relative to the presented pair, so it must be mapped back to candidate identity when comparisons are reused. Crucially, c_{ij} concerns this particular judgment; confidence values from unrelated pairs cannot be treated as a global ranking of candidates.

2.2 System architecture: prediction followed by verification

ForeAgent builds on AIDE's search over program nodes. Executed nodes provide code, runtime observations, and measured performance that can inform later drafting, debugging, or improvement. ForeAgent modifies the *improvement* stage by inserting prediction-based filtering between generation and execution (Zheng et al., 2026, Section 6.2):

Executed parent $\longrightarrow$ Candidate generation $\longrightarrow$ LLM comparisons
$\longrightarrow$ Confidence-gated selection $\longrightarrow$ Execution and feedback.

The published setup proposes $m = 10$ improvements to a parent solution, applies a confidence threshold $\tau = 0.7$, and selects the top rank group with $k = 1$. The selected candidates are subject to the execution policy described below. Actual execution trains and evaluates a candidate program; it is distinct from calling the LLM to compare code. Successful verification returns measured feedback to the search process, so the full agent retains interaction with its computational environment.

The predictor is grounded in task and dataset context rather than code alone. The paper constructs its data reports through a profiling, verification, and verbalization pipeline: generated analysis code is executed, the resulting logs are checked, and the observations are turned into a textual report (Zheng et al., 2026, Section 3.4). The report provides information with which the LLM can assess whether a pipeline suits the data. It is part of the prediction input, not the candidate's hidden evaluation score. For the offline baseline, the released reports are reused rather than regenerated.

2.3 Inference-time selection and execution rules

The core method uses pretrained LLM inference without fitting an additional reward model or updating model parameters. Accordingly, the relevant technical specification is its decision rule, not a new training loss. The confidence gate accepts a comparison when

$$a_{ij} = \mathbf{1}\{c_{ij} \geq \tau\}, \qquad \tau = 0.7. \tag{3}$$

A high-confidence winner can advance; an inconclusive judgment does not immediately establish a loser. This is a gate on self-reported confidence, rather than a calibrated probability model fitted by this project.

Tournament aggregation. In the fixed released implementation, each round compares adjacent active candidates; an odd candidate receives a bye. Confident winners advance and losers are eliminated. Inconclusive pairs enter a pending list. Pending candidates are compared with randomly chosen advanced candidates; if none has advanced, one pending candidate advances directly. A pending winner advances without removing its already-advanced opponent; another inconclusive outcome also advances the pending candidate. The process stops at one survivor or after the round count exceeds $2m$. Final ordering prioritizes survivors, then wins and later rounds; equal consecutive $(\text{wins}, \text{round})$ groups share a displayed rank (ForeAgent authors, 2026a).

Selection and probabilistic execution. Let r_i be the zero-based displayed rank. The multi-candidate branch marks $r_i < k$ candidates eligible and independently applies an execution probability $p_{\text{exec}} = 0.2$ to each. If these tests produce an empty execution set, the original code forces execution of the first rank-0 candidate in sorted order (ForeAgent authors, 2026a). Formally, let $B_i \sim \text{Bernoulli}(p_{\text{exec}})$ and let i_0 index that first rank-0 candidate. The preliminary and final execution indicators are

$$q_i = \mathbf{1}\{r_i < k\}\, B_i, \qquad e_i = q_i + \mathbf{1}\left\{\sum_j q_j = 0\right\} \mathbf{1}\{i = i_0\}. \tag{4}$$

Thus the original multi-candidate policy executes at least one candidate, while ties may permit several executions. The probability gate and mandatory fallback together specify the execution decision used in Section 3.

2.4 Baseline evaluation and the scope of this project

The paper evaluates two different capabilities. First, its *Data-centric Solution Preference* task measures how well an LLM predicts pairwise winners. The Preference Corpus contains 895 curated solutions and 18,438 pairs from 26 tasks spanning computer vision, natural language processing, and data science. The principal metric is micro-averaged pairwise accuracy,

$$\text{Acc}_{\text{pair}} = \frac{1}{N} \sum_{\ell=1}^{N} \mathbf{1}\{\widehat{y}_\ell = y_\ell\}, \tag{5}$$

where N counts the evaluated pairs and y_ℓ is the recorded better-candidate label. The paper reports 61.5% accuracy for DeepSeek-V3.2-Thinking, averaged over three full-corpus runs (Zheng et al., 2026, Table 2). This is a published reference result, not a result achieved by this project.

Second, the full ForeAgent system is evaluated against AIDE on five MLE-bench tasks under a 12-hour limit, with three runs per task. That evaluation measures final task performance through Beat Ratio, the proportion of human leaderboard contestants outperformed, alongside execution-time and search-breadth analyses (Zheng et al., 2026, Sections 6.3–6.4). Pairwise accuracy does not establish these end-to-end outcomes: a predictor can make locally correct comparisons yet select poorly within a candidate pool, and a good offline selection does not establish actual training-time savings.

This project treats the pairwise predictor and candidate-selection controller as related baseline components. Earlier exploratory prediction runs used a fixed sample of 246 pairs across 26 tasks. The frozen reproduction reported next focuses on the original ranking and complete execution-control logic, using released historical agent records. Its evaluation target is agreement with the recorded outputs, conditional on archived comparison responses and probability outcomes. This establishes the behavior of the original controller before subsequent selection-quality and ranking-improvement experiments.

3 Baseline Reproduction

3.1 Objective and frozen scope

This reproduction examines whether ForeAgent's released ranking and execution control logic reproduces its recorded historical outputs when supplied with the original candidate batches, generation order, and saved observations. The baseline is the original implementation at commit `c4d52cf`, with

the original cache implementation preserved (ForeAgent authors, 2026a). The comparison backend supplies recorded responses by batch position, and the execution branch receives the probability outcomes recorded in the logs. The resulting experiment is a *conditional historical-trace replay*: it verifies the state updates produced from those observations.

The experiment is frozen as `foreagent_original_historical_replay_v1`, dated November 7, 2026. The freeze record binds the source snapshots, input manifests, scripts, verification outputs, and archive using SHA-256 checksums. Subsequent cache corrections and alternative aggregation methods belong to separately versioned experiments. The frozen result in this section concerns the original control logic.

3.2 Released records and experimental setup

The inputs come from the authors' public `agent_runs/ForeAgent/` release, fixed at dataset revision `6b322cb` (ForeAgent authors, 2026b). All 15 released ForeAgent runs were inspected: three runs for each of five MLE-bench tasks. For each run, the inputs include its actual `config.yaml`, full `journal.json`, and full `aide.log`. These 45 original text files were checked against their recorded sizes and hashes. Table 1 summarizes recovery coverage.

Table 1: Coverage of the released historical runs. Logged batches include interrupted generation and batches with final-journal gaps. Each recovered batch contains ten candidates.

Task	Runs	Logged batches	Recovered	Candidates
Aerial Cactus Identification	3	14	12	120
Histopathologic Cancer Detection	3	5	4	40
Stanford COVID Vaccine	3	45	42	420
Statoil Iceberg Classification	3	60	57	570
Ventilator Pressure Prediction	3	16	14	140
Total	15	140	129	1,290

The saved configurations agree on the relevant settings: ten improvement candidates, confidence threshold 0.7, Top-$k = 1$, execution probability 0.2, and a 12-hour agent time limit. The comparison-model label is DeepSeek-V3.2-Thinking with temperature 1.0. These model settings describe the source runs; this replay consumes their saved responses. Local callbacks record execution decisions, while training and inference observations come from the archived runs. Additional model API calls and candidate-training runs both equal zero.

Recovery uses record availability as the eligibility criterion. Of 140 logged batches, 129 have completed comparison histories and corresponding final-journal nodes. The remaining 11 are retained in a separate coverage ledger: six end during generation, and five contain generated nodes absent from the final journal. This rule was applied using structural completeness, independently of candidate scores or agreement results.

3.3 Recovery and replay procedure

Candidate identity, grouping, and order. Messages of the form `Generated improvement candidate i/10` identify the original generation order. Candidate IDs are resolved to their full journal entries and actual source code. The `node2parent` map verifies a shared parent within each recovered batch. Creation-time ordering provides a second check on the logged sequence. The journal's stored list reflects post-selection saving and re-appending, so generation logs and timestamps establish the replay input order.

Historical comparisons. Each candidate in a batch stores the same `wm_comparison_results` sequence, containing winner position, confidence, round, and validity. A local provider supplies these archived results to the original `tournament_rank()` calls in their recorded order. The replay checks response counts, round indices, and agreement between confidence and the 0.7 validity gate. It also checks 8,725 ordinary comparisons against the logged participant IDs and outcomes, and 15,148 revival comparisons against the logged pending-candidate identities and outcomes.

The original controller's random seed or RNG state remains an open recovery item. For revival matches, the advanced opponent is represented by an explicitly marked placeholder choice generated with demonstration seed 0. Responses are supplied by recorded batch position. The checked historical information identifies the pending candidate and outcome; it supplies partial evidence about the advanced opponent. Consequently, the agreement claim concerns control-flow replay conditional on the recorded responses, with exact original revival-pair reconstruction reserved for further log analysis.

Execution decisions and mandatory fallback. The original execution statements are extracted directly from `agent.py`, lines 700–767, and checked for AST equality. Logged probability-trigger outcomes drive the eligible candidates' execution tests. The original code then applies its mandatory fallback: whenever this stage selects an empty execution set, the first rank-0 candidate in sorted order is executed. Local callbacks record selected identities. The resulting execution set is checked against journal `is_skipped` flags, and its order is checked against execution-parsing log messages. Supplying observed probability outcomes conditions this check on the historical draws; the verified behavior is Top-k handling, candidate selection, and fallback processing.

3.4 Results and agreement with the released traces

For all 129 recovered batches, the original implementation reproduces the recorded candidate ranks, round metadata, sorted order, and final execution decisions. Table 2 reports the comparison units explicitly. Across these batches, the replay consumes 23,873 comparison records, including repeated rounds and cached observations. These records are observations from the historical trajectory, with repeated comparisons contributing to the recorded control flow.

Table 2: Agreement conditional on archived comparison responses and logged probability outcomes. Candidate-level and batch-level denominators measure different units.

Checked output	Agreement	Rate
Candidate displayed rank	1,290 / 1,290	100%
Candidate round metadata	1,290 / 1,290	100%
Complete sorted candidate order	129 / 129 batches	100%
Final execution set	129 / 129 batches	100%
Final execution order	129 / 129 batches	100%

Mandatory execution fallback occurs in 87 of the recovered batches, matching the log events. For example, in the first recovered batch of the Aerial Cactus run with UUID prefix `c5d8f2ae`, candidate `69daaf39` receives rank 0. Its probability test is recorded as skipped, after which the original fallback selects that candidate. Both the resulting execution identity and the final journal flag agree with the replay.

The 100% values represent *historical-output agreement*. The recorded winner positions are supplied as observations; true candidate-performance scores play no role in recovering ranks. Thus the result establishes faithful processing of these observed histories. Pairwise prediction accuracy, selection quality against executed scores, and end-to-end agent efficiency are separate evaluation questions.

3.5 Interpretation and reproducibility artifacts

The replay establishes an implementation baseline for the candidate-selection component: the recovered historical observations lead to the same ranking and execution outputs under the released control rules. The paper's full-corpus prediction accuracy and its five-task online performance results provide broader reference experiments, while this section contributes a direct consistency check against the released agent trajectories (Zheng et al., 2026). The archived responses already reflect the source runs' inference and cache behavior; fresh model judgments and original revival-opponent reconstruction require their own experimental conditions.

The accompanying `baseline_freeze.json` pins the source, data, scripts, and verified outputs by SHA-256. The archive contains full logs and journals, the coverage ledger, and per-batch comparisons. Its `replay_author.py` and `verify_and_report.py` scripts reproduce and check these results.

References

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-Driven Exploration in the Space of Code. *arXiv:2502.13138*, 2025. https://arxiv.org/abs/2502.13138.

Jingsheng Zheng, Jintian Zhang, Yujie Luo, Yuren Mao, Yunjun Gao, Lun Du, Huajun Chen, and Ningyu Zhang. Can We Predict Before Executing Machine Learning Agents? In *Proceedings of ACL 2026 (Volume 1: Long Papers)*, pp. 3941–3974, 2026. https://aclanthology.org/2026.acl-long.182/.

Jun Shern Chan et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. *arXiv:2410.07095*, 2024. https://arxiv.org/abs/2410.07095.

ForeAgent authors. Released implementation of Predict Before Execute. GitHub repository, fixed commit c4d52cf (full identifier in the URL below). Relevant files: world_model.py, agent.py, and config.yaml under mle-bench/agents/aide/. https://github.com/zjunlp/predict-before-execute/tree/c4d52cf99bd870d830b456ac7c0684aec1aef375.

ForeAgent authors. PredictBeforeExecute: released ForeAgent execution records. Hugging Face dataset, fixed revision 6b322cb; subdirectory agent_runs/ForeAgent/. https://huggingface.co/datasets/zjunlp/PredictBeforeExecute/tree/6b322cb88bdbcb2b2d3897ec7d0ded94a5bb2d06/agent_runs/ForeAgent.

Generative AI use. OpenAI Codex assisted with source and log inspection, Python replay and verification tooling, English drafting, and LaTeX typesetting. Requests included (translated from Chinese): "Replay the original version using the authors' real candidate groups, order, and saved judgments, and check agreement" and "Freeze this part and add it to HW3's Baseline Reproduction using the previous NeurIPS template." Original prompts and affected components are recorded in the accompanying AI_USAGE.txt.